

PROJECT REPORT

STUDENT MANAGEMENT SYSTEM

A Python-based command-line application for student records, attendance, grades and basic calculations.

Student Name	SOUMYA SINGH
Registration No.	26BCE10773
Programme / Branch	CSE CORE

1. Introduction

The Student Management System is a simple Python-based project designed to manage basic student information. The supplied project allows a user to add students, manage attendance, assign grades, search for student records, delete students, view reports and perform basic calculations. The project is implemented as a local, single-user command-line application using Python and an in-memory dictionary.

The report is organized according to the VITyarthi project-report structure, which requires a cover page, introduction, problem statement, requirements, architecture and design diagrams, implementation, results, testing, challenges, learnings, future enhancements and references.

2. Problem Statement

The project statement identifies a common problem: teachers may track attendance and grades using paper registers or scattered spreadsheets, making it slower to identify attendance concerns and increasing the risk of records being miscounted or forgotten between sessions. The proposed project provides one local command-line tool for maintaining a class roster, recording attendance, storing grades and viewing student information.

2.1 Scope of the Project

The project is scoped as a single-user, local tool. It covers student records, attendance tracking, grade recording, search/delete operations, reporting and a calculator. It does not cover multi-teacher accounts, timetabling or networked access.

2.2 Target Users

- School or college teachers who want to manage their own student attendance and grade records.
- Students using the project to track their own attendance and grades.

3. Objectives

- Maintain student records through a simple command-line interface.
- Record present/absent attendance for each student.
- View student records containing attendance counts and grade.
- Search for an individual student's attendance percentage and grade.
- Warn the user when a searched student's attendance percentage is below 75%.
- Delete an existing student record.
- Store a numeric grade from 0 to 100 and classify it from A to F.
- Provide seven basic calculator operations.
- Demonstrate Python fundamentals including dictionaries, loops, conditional statements, input/output and arithmetic operators.

4. Functional Requirements

ID	Requirement	Description
FR-01	Add Student	Accept a student name and create a new student record.
FR-02	Duplicate Student Check	Inform the user if the entered student name already exists.

FR-03	Mark Attendance	Accept P for present or any other response for absent; increase total attendance by one.
FR-04	View Report	Display each student's name, present count, total attendance and grade.
FR-05	Search Student	Search by name and display attendance, percentage and grade.
FR-06	Attendance Warning	Display a warning when attendance percentage is below 75%.
FR-07	Delete Student	Delete an existing student record.
FR-08	Calculator	Support addition, subtraction, multiplication, division, exponentiation, modulus and floor division.
FR-09	Set Grade	Accept and store a grade from 0 to 100.
FR-10	Grade Classification	Display A, B, C, D, E or F according to the entered score.
FR-11	Exit	Terminate the application when option 8 is selected.

5. Non-functional Requirements

Requirement	Project Specification
Usability	Numbered menu, clear prompts and readable text output.
Performance	Dictionary lookup is suitable for a small number of classroom records and operations should execute immediately.
Portability	The project is intended for Python and requires no third-party Python libraries.
Maintainability	Student data uses a consistent dictionary structure and menu choices are separated through conditional branches.
Reliability	The program checks student existence before search, attendance, deletion and grade updates and validates the grade range.
Data Persistence	The current implementation does not persist data; records are lost when the application terminates.

6. System Architecture

The supplied architecture describes a single-process console application with no external services, database, network communication or external Python packages. Data is stored only for the duration of the program session.

System Architecture



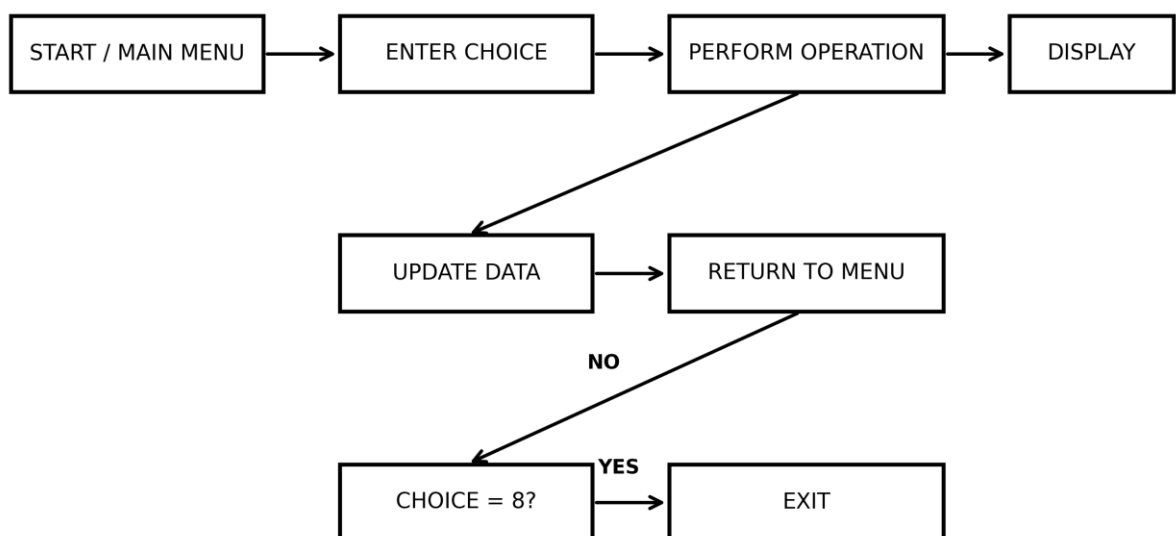
Architecture processing flow:

1. Start application.
2. Create an empty student dictionary.
3. Display the main menu.
4. Accept the user's choice.
5. Execute the selected operation.
6. Return to the main menu.
7. Exit when option 8 is selected.

7. Design Diagrams

7.1 Workflow Diagram

Student Management System - Workflow



The main workflow returns to the menu after each operation. An invalid choice displays an error and also returns to the menu.

7.2 Use Case Diagram / Specification

Primary Actor: User / Student Record Administrator

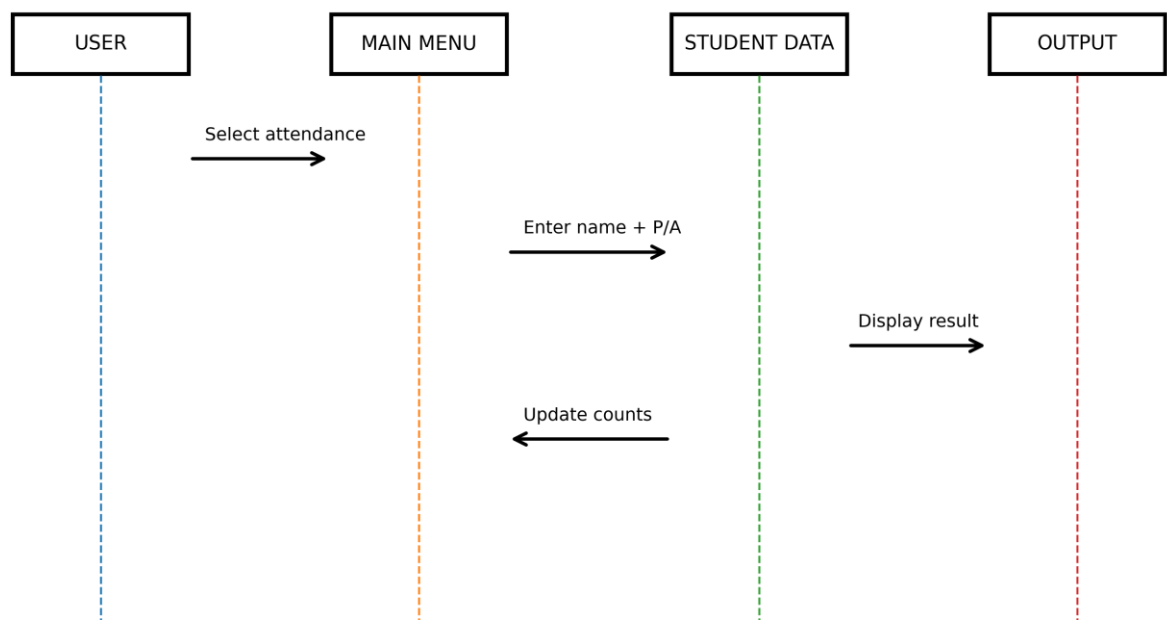
ID	Use Case	Description
UC-01	Add Student	Add a new student name
UC-02	Mark Attendance	Record present/absent status
UC-03	View Report	Display all student records
UC-04	Search Student	Display one student's details
UC-05	Delete Student	Remove a student
UC-06	Calculator	Perform arithmetic operations
UC-07	Set Grade	Store and classify a grade
UC-08	Exit	Close the application

Typical Search Flow:

8. User selects Search Student.
9. User enters the student name.
10. System checks the dictionary.
11. If found, the system calculates attendance percentage.
12. The system displays attendance and grade.
13. If attendance is below 75%, a warning is displayed.
14. If the student is not found, the system displays 'Student not found.'

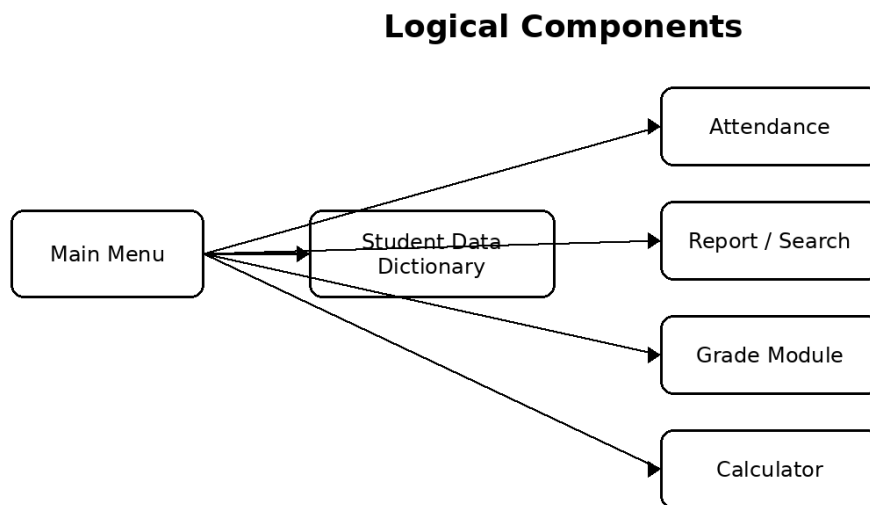
7.3 Sequence Diagram

Sequence Diagram - Mark Attendance



The project documentation defines sequences for Add Student, Mark Attendance, Search Student and Set Grade. The diagram above summarizes the Search Student interaction.

7.4 Component Diagram



The project is implemented in a single app.py file. The component diagram therefore represents logical components rather than separate Python modules or classes.

7.5 Storage Design

The project does not use a database or persistent file storage. The main in-memory structure is:

```
student = {
    "Student Name": {
        "present": 0,
        "total": 0,
        "grade": 0.0
    }
}
```

Each student name is the dictionary key. The associated nested dictionary stores present count, total attendance count and numeric grade.

8. Design Decisions & Rationale

Console Interface: Selected because it is simple to implement and suitable for a beginner Python project.

Dictionary-Based Storage: Student names are used as keys for direct lookup.

Nested Student Records: Attendance and grade fields remain grouped with the relevant student.

Menu-Driven Design: A numbered menu clearly presents available operations and allows repeated use without restarting.

In-Memory Storage: The current project intentionally avoids database/file storage to focus on Python fundamentals.

Separate Functional Options: Student, attendance, reporting, searching, deleting, calculator and grading are represented as separate menu choices.

Grade Thresholds: The thresholds follow the rules implemented in app.py.

9. Implementation Details

9.1 Technology and Tools

- Python
- Python Dictionary
- Git
- GitHub
- Markdown

9.2 Main Data Structure

```
student = {}
```

Each key is a student name and each value contains present, total and grade fields.

9.3 Main Control Flow

The program uses while True to repeatedly display the main menu until the user selects Exit.

9.4 Attendance Calculation

```
percentage = (present / total) * 100
```

If no attendance has been recorded, the percentage is set to 0.

9.5 Grade Logic

Score Threshold	Grade
90 or more	A
80 or more	B
70 or more	C
60 or more	D
40 or more	E
Below 40	F

9.6 Calculator

The calculator accepts float values and supports addition, subtraction, multiplication, division, exponentiation, modulus and floor division.

9.7 Input Validation

- Student existence is checked before student-specific operations.
- Grade input is checked against the 0–100 range.
- Main-menu and calculator choices are checked for valid options.

The supplied project documentation explicitly records that non-numeric input errors and division/modulus/floor-division by zero are not currently handled.

10. Screenshots / Results

The following screenshots are taken directly from the supplied project ZIP and show the actual application interactions.

```
==Main Menu==  
1. Add the name of the student  
2. Mark Attendance  
3. View Report  
4. Search Student  
5. Delete Student  
6. Calculator  
7. Set Grade  
8. Exit
```

Figure 1 – Main Menu

```
==Main Menu==  
1. Add the name of the student  
2. Mark Attendance  
3. View Report  
4. Search Student  
5. Delete Student  
6. Calculator  
7. Set Grade  
8. Exit  
Enter choice: 1  
Enter student name: Rahul  
Rahul Added.
```

Figure 2 – Adding a Student

```
Enter choice: 2
Students: ['Arnav', 'Aditi', 'Rahul', 'Priya']
Enter student name: Rahul
Is the student present today? (y/n): n
Attendance for Rahul marked as ABSENT.
```

Figure 3 – Marking Attendance

```
Enter choice: 3
```

Name	Present	Total	Grade
Arnav	2	2	95.00
Aditi	0	2	89.00
Rahul	1	2	85.00
Priya	2	2	54.00

Figure 4 – Viewing Report

```
Enter choice: 4
Enter student name to search: Rahul
Name: Rahul
Present: 1
Total: 2
Percentage: 50.00%
Grade: 85.00
Warning: Attendance is below 75%
```

Figure 5 – Searching for a Student

```
Enter choice: 7
Enter student: Rahul
Enter student's grade (0-100): 85
Grade set for Rahul : 85.00
Grade: B, Good
```

Figure 6 – Setting a Grade

```
Enter choice: d
Enter first number: 6778
Enter second number: 39834
Result: 0.17015614801425918
```

Figure 7 – Using the Calculator

```
Enter choice: 5
Students: ['Arnav', 'Aditi', 'Rahul', 'Priya']
Enter student name to delete: Aditi
Aditi Deleted Successfully.
```

Figure 8 – Deleting a Student

11. Testing Approach

Test environment: Python 3.x, console/terminal, with no external dependencies.

Test ID	Feature	Test Input	Expected Result
T01	Add student	1, Rahul	Rahul is added
T02	Duplicate student	Add Rahul again	Already Exists message
T03	Attendance present	2, Rahul, P	Total +1 and Present +1
T04	Attendance absent	2, Rahul, A	Total +1, Present unchanged
T05	View report	3	Student records are displayed
T06	Search existing	4, Rahul	Student details and percentage shown
T07	Search missing	4, Amit	Student not found
T08	Delete existing	5, Rahul	Student is deleted
T09	Delete missing	5, Amit	Student not found
T10	Grade valid	7, Rahul, 85	Grade B is displayed
T11	Grade invalid	7, Rahul, 105	Range error is displayed
T12	Calculator addition	6, a, 10, 5	Result 15
T13	Calculator subtraction	6, b, 10, 5	Result 5
T14	Calculator multiplication	6, c, 10, 5	Result 50
T15	Calculator division	6, d, 10, 5	Result 2
T16	Calculator exponentiation	6, e, 2, 3	Result 8
T17	Calculator modulus	6, f, 10, 3	Result 1
T18	Calculator floor division	6, g, 10, 3	Result 3
T19	Exit	8	Application exits

11.1 Edge Cases

- Search for a student before any student has been added.
- Search a student with zero attendance.
- Enter grade 0 and grade 100.
- Enter a grade below 0 or above 100.
- Enter an invalid menu option.
- Attempt calculator division by zero.

11.2 Current Testing Limitations

The project documentation identifies the following current limitations:

- Non-numeric input where float() is required is not explicitly caught.
- Division, modulus and floor division by zero are not explicitly handled.
- Attendance input is not case-insensitive; p is treated as absent.

12. Challenges Faced

- Keeping multiple menu operations within one continuously running program.
- Maintaining attendance counters correctly.
- Calculating attendance percentage without dividing by zero when there is no attendance.
- Validating grades and mapping them to A–F categories.
- Handling missing students for search, attendance, deletion and grading operations.
- Keeping the project simple while still providing multiple useful functions.

13. Learnings & Key Takeaways

- Practical use of Python dictionaries and nested dictionaries.
- Use of while loops for interactive programs.
- Use of conditional statements for decision-making.
- Use of input validation for student existence and grade range.
- Implementation of add, search, view and delete operations.
- Calculation and display of attendance percentage.
- Designing a small real-world application around Python fundamentals.
- Using Markdown documentation and screenshots alongside source code.
- Understanding the importance of documenting current limitations and test cases.

14. Future Enhancements

- Add persistent storage using a file or database.
- Add unique student IDs or registration numbers.
- Add edit/update functionality for student details.
- Add monthly or subject-wise attendance.
- Add automatic grade calculation from multiple assessments.
- Add login/authentication for administrators.
- Add a graphical user interface.
- Improve input validation for non-numeric values and division-by-zero cases.
- Make attendance input case-insensitive.

15. References

1. VITYarthi – Build Your Own Project: General Project Instructions & Submission Guidelines
2. Chatgpt and Google